

Vector Embedded MemAcc

Technical Reference

Contents

1	Introduction	6
1.1	Architecture Overview	6
2	Functional Description	8
2.1	Features	8
2.1.1	Memory Address Translation	8
2.1.2	Job Management and Processing	9
2.1.3	Parallel Processing of Different Asynchronous Jobs	10
2.1.4	Splitting of Memory Driver Access Requests	10
2.1.5	Memory Driver handling	12
2.1.6	64-bit Address Space Support	12
2.1.7	Multi Binary	12
2.1.8	NAND-Flash Memory Support	15
2.1.9	Read only mode for specific memory areas	18
2.2	Error Handling	20
2.2.1	Development Error Reporting	20
2.3	Compliance	20
3	Integration	22
3.1	Embedded Implementation	22
3.2	Initialization	22
3.2.1	Single Binary Use Case	22
3.2.2	Multi Binary Use Case	22
3.3	Main Functions	23
3.3.1	Single Binary Use Case	23
3.3.2	Multi Binary Use Case	24
3.4	Synchronization	24
3.4.1	Critical Sections	24
3.5	Memory Sections	25
3.5.1	Multi Binary use case	25
3.6	Linker Symbols for Multi Binary use case	26
3.7	Callouts	26
3.8	Error Reporting Det	27
3.9	INDIRECT_STATIC Mems	27
3.10	INDIRECT_DYNAMIC Mems and their dynamic activation	27
3.11	Handling of ECC Errors	30
3.12	Compatibility with former AUTOSAR releases	30

4	API Description	31
4.1	Type Definitions	31
4.2	Provided Functions	31
4.2.1	MemAcc_InitMemory	31
4.2.2	MemAcc_Init	31
4.2.3	MemAcc_DelInit	32
4.2.4	MemAcc_GetVersionInfo	32
4.2.5	MemAcc_GetJobResult	32
4.2.6	MemAcc_GetJobStatus	33
4.2.7	MemAcc_GetMemoryInfo	33
4.2.8	MemAcc_GetProcessedLength	34
4.2.9	MemAcc_GetJobInfo	34
4.2.10	MemAcc_MainFunction	35
4.2.11	MemAcc_Cancel	35
4.2.12	MemAcc_Read	36
4.2.13	MemAcc_Write	36
4.2.14	MemAcc_Erase	37
4.2.15	MemAcc_Compare	37
4.2.16	MemAcc_BlankCheck	38
4.2.17	MemAcc_HwSpecificService	39
4.2.18	MemAcc_RequestLock	39
4.2.19	MemAcc_ReleaseLock	40
4.2.20	MemAcc_ActivateMem	41
4.2.21	MemAcc_DeactivateMem	41
4.3	Required Functions	42
4.4	Configurable Interfaces	43
4.5	Service Ports	44
5	Configuration	45
5.1	Configuration Variants	45
5.2	Configuration with DaVinci Configurator Classic	45
5.2.1	Memory Address Mapping	45
5.2.2	Parallel Processing Configuration	46
5.2.3	Multi Binary Configuration	46
5.2.3.1	Redirected requests	47
5.2.4	NAND-Flash Configuration	49
5.2.5	Read only mode for specific memory areas	50
5.2.6	Unaligned Read and Compare Support	50

6	Glossary and Abbreviations	51
7	Contact	53

Document Information

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	Specification of Memory Access	R23-11
[2]	AUTOSAR	Specification of Memory Driver	R23-11
[3]	AUTOSAR	Specification of Development Error Tracer	V3.2.0
[4]	VECTOR	Memory Mapping in MICROSAR Classic General Overview User Manual	see delivery
[5]	VECTOR	MICROSAR Classic MemMap Technical Reference	see delivery
[6]	Micron Technology Inc	TN-29-59: Bad Block Management in NAND Flash Memory Introduction	Rev. H 4/11 EN
[7]	Vector	MICROSAR vMem Technical Reference	see delivery



Caution

We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector's release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

1 Introduction

This document describes the functionality, integration, API and configuration of the component MemAcc.

The MemAcc is specified in AUTOSAR [1].

This module coordinates the memory access by multiple users to avoid conflicts with shared memory resources.

The basic functionalities offered by the MemAcc to access the memory are read, write, erase and blank check. Note that the MemAcc does not access the memory directly itself, but forwards requests to the underlying Mem drivers [2].

To manage the memory access, the MemAcc implements the following main features:

- > To avoid clashes within the memory resource the MemAcc processes only memory access jobs that do not interfere with each other. To be able to accept multiple jobs for different regions of memory the MemAcc provides a **queueing mechanism**. Furthermore, the order in which the jobs within the queue are being processed is **based on priorities**.
- > To offer higher layers independence from the actual memory distribution within the underlying Mem driver modules, the MemAcc performs a memory address mapping. Upper layers only use logical addresses to access memory. The logical memory is structured in so-called **address areas**. The MemAcc performs the mapping from **logical addresses** to **physical addresses** and forwards requests to the underlying Mem driver modules.
- > To enable higher system response time for higher job priority execution, the MemAcc divides the asynchronous jobs up into jobs of smaller length (**job-splitting**), which are forwarded to the underlying Mem drivers. This way the related Mem driver is only blocking for a short time before a different asynchronous job with higher priority could be started. The length of the forwarded job depends on the specific request and configuration.

1.1 Architecture Overview

These are the interfaces of the module:

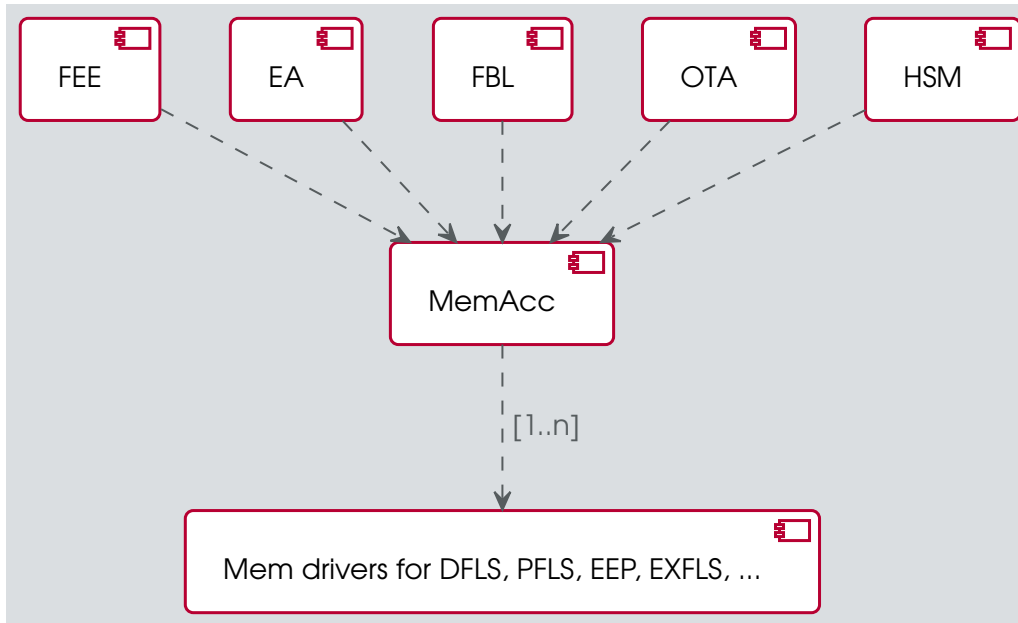


Figure 1.1 Module Architectural Environment

2 Functional Description

2.1 Features

2.1.1 Memory Address Translation

One of the main responsibilities of the MemAcc is the mapping between physical and logical address space, to provide an abstraction of the available memory resources. Upper layers only need to use logical addresses and do not have to be concerned with data spreading and storage on different memory devices. Whereas the underlying memory driver can only handle the actual physical addresses.

The physical memory on the level of the Mem drivers is structured in sectors, the smallest erasable units. Identical and coherent sectors can be grouped into sector batches. This way a greater number of identical, contiguous sectors can be configured collectively and handled in the same way. In the MemAcc the entire physical memory of all available sector batches is structured in logical memory areas, the address areas. An address area contains sub address areas and each sub address area represents a continuous part of a physical sector batch. In other words, a sub address area is a part of an address area, which corresponds to a physical continuous area of memory.

For every sub address area a logical start address is configured. The sub address areas must not overlap within the address area. Multiple sub address areas can target the same physical memory. The length of a sub address area is determined by the number of sectors it represents and their sector size. Note that different address areas may have the same logical addresses.

Within every asynchronous job request (except hardware specific job) an address area identifier, a logical address within the respective address area and a length has to be passed to the MemAcc. The MemAcc maps the passed logical address to a physical address (Mem device instance and device specific address). Note that the logical address space may contain gaps in front, between, and after sub address areas. Jobs may span multiple sub address areas, but must never start in, end in, or contain gaps.

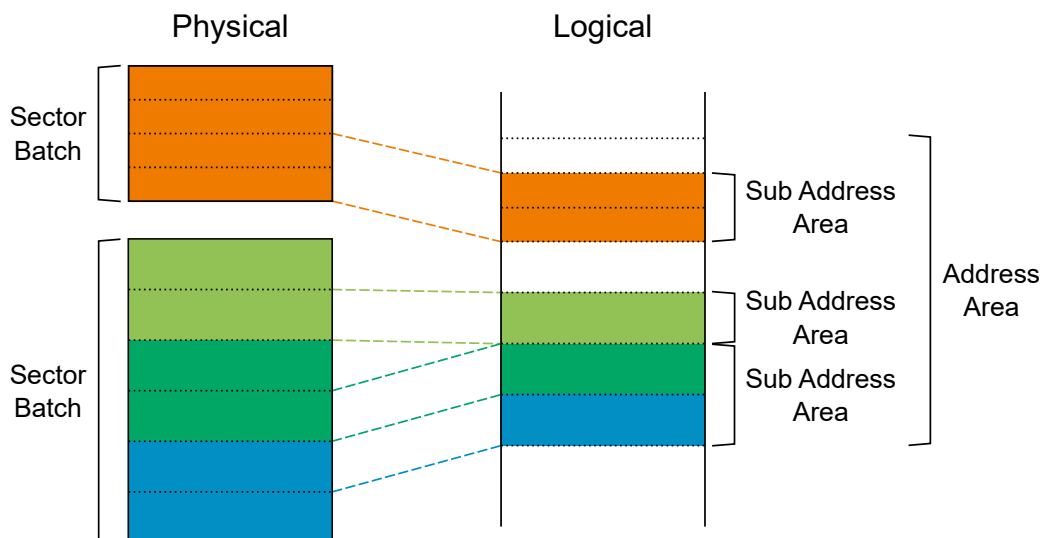


Figure 2.1 overview memory address translation scheme

2.1.2 Job Management and Processing

The MemAcc accepts at most one asynchronous job request per address area at a time. Since the MemAcc will process only one asynchronous job per address area at a time, it implements a queue with the size of the number of address areas. An asynchronous job request is only accepted and queued if there is no other currently pending job for the requested address area. So, if for instance a read job for an address area is queued, an erase job for the same address area will not be accepted before the read job is finished.

Each address area has its priority which can be configured. The priorities 0 (lowest priority) until 65535 (highest priority) can be selected for an address area.

An asynchronous job on an address area with higher priority is processed before asynchronous jobs on address areas with lower priority. If there is an already ongoing asynchronous job on an address area with lower priority, it will be suspended as soon as possible. There are two different scenarios possible:

1.Scenario: If the Mem driver provide suspend and resume functions in accordance with the AUTOSAR R24-11 specification or later versions, and `/MICROSAR/MemAcc/MemAccGeneral/MemAccSuspendResumeEnabled` is set to true: Lower priority jobs are suspended immediately after a higher priority job was set up.

2.Scenario: If the Mem driver provide no suspend and resume functions, or provide them in accordance with the AUTOSAR R23-11 specification or previous versions, and `/MICROSAR/MemAcc/MemAccGeneral/MemAccSuspendResumeEnabled` is set to false: Lower priority jobs are suspended after the currently processed job step is finished.



Note

In AUTOSAR R23-11 and previous versions the `<Mem>_Suspend/ <Mem>_Resume` functions are ill-defined. Both functions are specified with void instead of standard return values in these versions. As there are multiple Mem drivers that either do not support this feature on a hardware level or suspend operations are otherwise limited (e.g. multiple suspensions may not be supported), the function calls can not safely be used without confirmation on whether the function call actually worked. Therefore, MemAcc does not support suspend and resume functions based on these older versions. If Mem driver provide suspend and resume functions with void return value, the feature has to be disabled with the `/MICROSAR/MemAcc/MemAccGeneral/MemAccSuspendResumeEnabled` configuration. Only enable this feature if all Mem drivers suspend and resume functions signature are according to version R24-11 or later.

The internal processing of the `MemAcc_MainFunction()` is optimized in a way, that the time between two `MemAcc_MainFunction()` shall be “used” effectively by the underlying hardware to process an asynchronous job. Therefore, after a queued job or one of its job steps is finished and a following job or job step is still in the queue, it will be forwarded to the

Mem/hardware within the same `MemAcc_MainFunction()` cycle. With this behavior, the overall required number of `MemAcc_MainFunction()` calls and therefore the time needed to process and finish a job is minimized.

If a read, compare, write or erase Mem job step fails, it will be automatically retried a set number of times. The amount of retries can be configured.

The MemAcc provides the current job processing status information via `MemAcc_GetJobResult()` and the job result of the last finished job via `MemAcc_GetJobResult()`. In case a job end notification function is configured, MemAcc notifies the upper layers by calling the configured notification function (see *Configurable Interfaces*).

2.1.3 Parallel Processing of Different Asynchronous Jobs

The module provides a mechanism to allow parallel processing of jobs on lower layers, so-called synchronization groups. The user can configure sub address areas, which shall be synchronized. These sub address areas must be distributed to different synchronization groups, see also *Parallel Processing Configuration*. In this group, only one job can be executed at a time. In case synchronization groups are created, all sub address areas must be distributed to the groups.

This mechanism can be used to allow parallel processing of a job operating on the internal flash and simultaneously executing a job on an external flash. Otherwise, access to an external flash blocks all internal jobs as well.



Note

This feature can only be used for different address areas. Multiple synchronization groups that are part of a single address area will not be parallelized.

2.1.4 Splitting of Memory Driver Access Requests

To avoid blocking requests the MemAcc splits asynchronous job requests into smaller jobs which are forwarded to the underlying Mem modules. An asynchronous job for an address area will be divided into at least the number of involved sub address areas, since each sub address area might correspond to an entirely different sector batch. But even requests which involve only one sub address area are divided into job steps of shorter length. The step length depends on the request and configuration parameters. In the following the partitioning of the individual asynchronous jobs is described in more detail.

Read and Compare

The MemAcc divides the read and compare request into smaller jobs which are forwarded to the underlying Mem modules. The maximum number of bytes the MemAcc module requests within one Mem read request is configured within the target Mem driver as `MemMaxRead`. If the remaining read and compare request length is smaller than `MemMaxRead` it is always ensured that requests are forwarded to the Mem driver which are aligned to `MemMinRead`.

**Caution**

As the `MemMaxRead` parameter is currently placed within the optional `MemBurstSettings` container, for performance reasons it is strongly recommended to set this value. If `MemMaxRead` is not set to a reasonably high value, many `MainFunctions` cycles might be needed for read requests.

Write

Write jobs are partitioned into smaller jobs of length `MemWritePageSize` as configured within the target Mem driver. If the Mem driver has the optional `MemBurstSettings` configured and the feature is enabled for the respective sub address area, the MemAcc can partition the job into bigger job steps of length `MemWriteBurstSize`. A Mem driver request with `MemWriteBurstSize` will be only forwarded if the remaining job length is at least `MemWriteBurstSize` and the start address of the respective job step is also aligned to `MemWriteBurstSize`. The MemAcc ensures that requests forwarded to the Mem driver are always aligned to the `MemWritePageSize` or the `MemWriteBurstSize`, respectively.

Blank Check

Blank check jobs are also partitioned into smaller jobs of length `MemWritePageSize` as configured within the target Mem driver. However, blank check jobs cannot utilize the burst feature and are never split into the larger `MemWriteBurstSize`, independent of the optional `MemBurstSettings` configuration. The MemAcc ensures that requests forwarded to the Mem driver are always aligned to the `MemWritePageSize`.

Erase

Erase jobs are partitioned into smaller jobs of length `MemEraseSectorSize` as configured within the target Mem driver. If the Mem driver has the optional `MemBurstSettings` configured and the feature is enabled for the respective sub address area, the MemAcc can partition the job into bigger job steps of length `MemEraseBurstSize`. A Mem driver request with `MemEraseBurstSize` will be only forwarded if the remaining job length is at least `MemEraseBurstSize` and the start address of the respective job step is also aligned to `MemEraseBurstSize`. The MemAcc ensures that requests forwarded to the Mem driver are always aligned to the `MemEraseSectorSize` or the `MemEraseBurstSize`, respectively.

Hardware Specific Requests will be forwarded directly to the target Mem driver.

**Note**

The MemAcc expects jobs with aligned addresses and length parameters. The only exception are unaligned read and compare jobs, see *Unaligned Read and Compare Support*.

2.1.5 Memory Driver handling

The module supports different Mem driver invocation modes configured within `MemAccSubAddressAreaConfiguration\MemAccMemInvocation`:

- > `DIRECT_STATIC` means the MemAcc will not call the `Mem_Init()` and `Mem_DeInit()` and will not trigger the Mem via `Mem_MainFunction()`. The Mem driver should be statically linked with the application and cannot be dynamically activated. This is the normal use case also optimized for performance as the Mem driver can be triggered faster than the MemAcc.
- > `INDIRECT_STATIC` means the MemAcc will call the `Mem_Init()` and `Mem_DeInit()` and will trigger the Mem via `Mem_MainFunction()`. The Mem driver should be statically linked with the application and cannot be dynamically activated. This mode might be used if the Mem driver initialization and processing needs to be handled by the MemAcc.
- > `INDIRECT_DYNAMIC` means the MemAcc will call the `Mem_Init()` and `Mem_DeInit()` and will trigger the Mem via `Mem_MainFunction()`. The Mem driver is linked as a separate binary and can only be dynamically activated (see *INDIRECT_DYNAMIC Mems and their dynamic activation*).

The `MemAccUseMemFuncPtrTable` parameter is not used. Whether the function pointer table is used is decided by the `MemAccSubAddressAreaConfiguration\MemAccMemInvocation`. Therefore,

- > for a `DIRECT_STATIC` Mem driver it is assumed that the Mem driver functions are called directly and not over the function pointer table. There is anyway no reason why a function pointer table should be provided for this mode.
- > for an `INDIRECT_STATIC/DYNAMIC` Mem driver it is assumed that the Mem driver functions are called using the Mem function pointer table. This is anyway necessary for a Mem which should be able to be deactivated/activated.

2.1.6 64-bit Address Space Support

Users can enable 64-bit address space support through configuration settings. By default, the address space remains at 32-bit. This enhancement allows for larger memory addressing capabilities, that benefits applications which require extensive memory management.

2.1.7 Multi Binary

The module supports the multi binary use case in addition to the specified AUTOSAR functionality. In general the MemAcc can be configured and used in different binaries, i.e. different DaVinci configuration projects. The most common use case probably is one MemAcc in application binary and one MemAcc in HSM binary flashed to the same controller.

By default, each MemAcc synchronizes access to the non volatile memory within its own binary, e.g. for the application binary access from the memory stack (Fee, Ea) and software download stack. In addition, for the multi binary use case the following two features are provided:

- > Access to the non volatile memory can also be synchronized between several binaries, which share the same memory hardware resources.
- > Memory access requests (Read, Write, Erase, BlankCheck and Compare but not HwSpecificService) can be redirected from a MemAcc of one binary to a MemAcc of another binary. For instance, a binary placed on a SOC with no access to memory, can forward a memory access request to a MemAcc of a binary with memory access.

To implement these features each MemAcc of each binary has a dedicated role, either master or satellite. There is always at least one master and there can be up to 15 satellites. The master is responsible for the synchronization between the binaries. The satellites first have to submit memory access requests to the master and may only perform the memory access once they have received permission from the master. Furthermore, they can redirect memory access requests to the master, if they don't have access to the memory themselves. The synchronization and forwarding of requests happens over shared memory (RAM) accessible by both master and satellites.

Combining the multi binary feature with the parallel processing of jobs (i.e. the usage of synchronization groups) allows the following use cases:

1. Both MemAcc master (e.g. in application binary) and MemAcc satellite (e.g. in HSM binary) have access to internal memory. This needs to be synchronized by the master. In addition the master has access to external memory, which can be done in parallel and therefore is configured within a separate synchronization group.

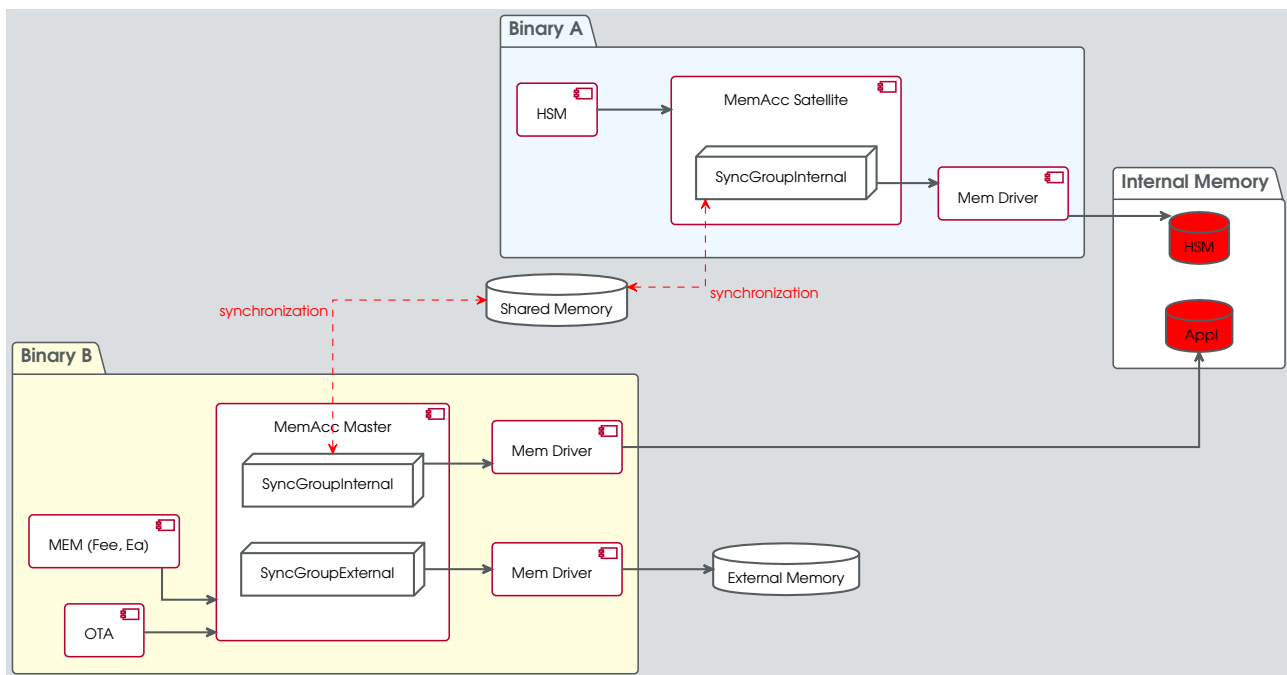


Figure 2.2 Use case 1

2. A MemAcc master has access to internal as well as external memory. A MemAcc satellite has no access to memory and needs to forward requests to the MemAcc master:

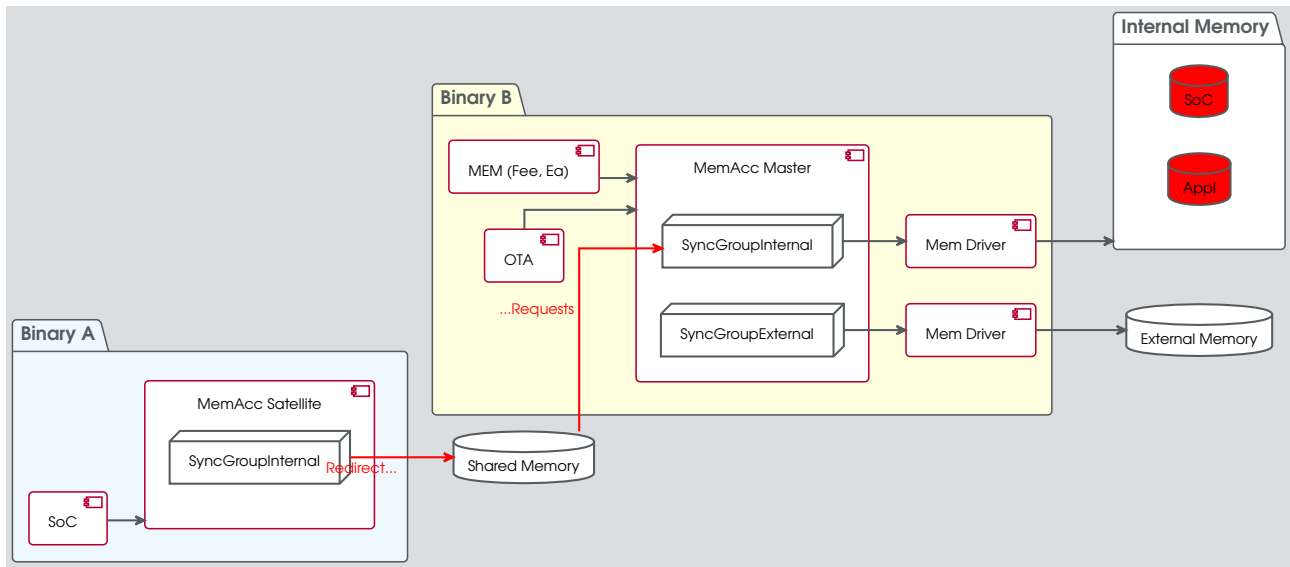


Figure 2.3 Use case 2

3. A MemAcc master has access to internal memory. A MemAcc satellite has access to internal memory but also wants to forward requests to the MemAcc master to extend memory:

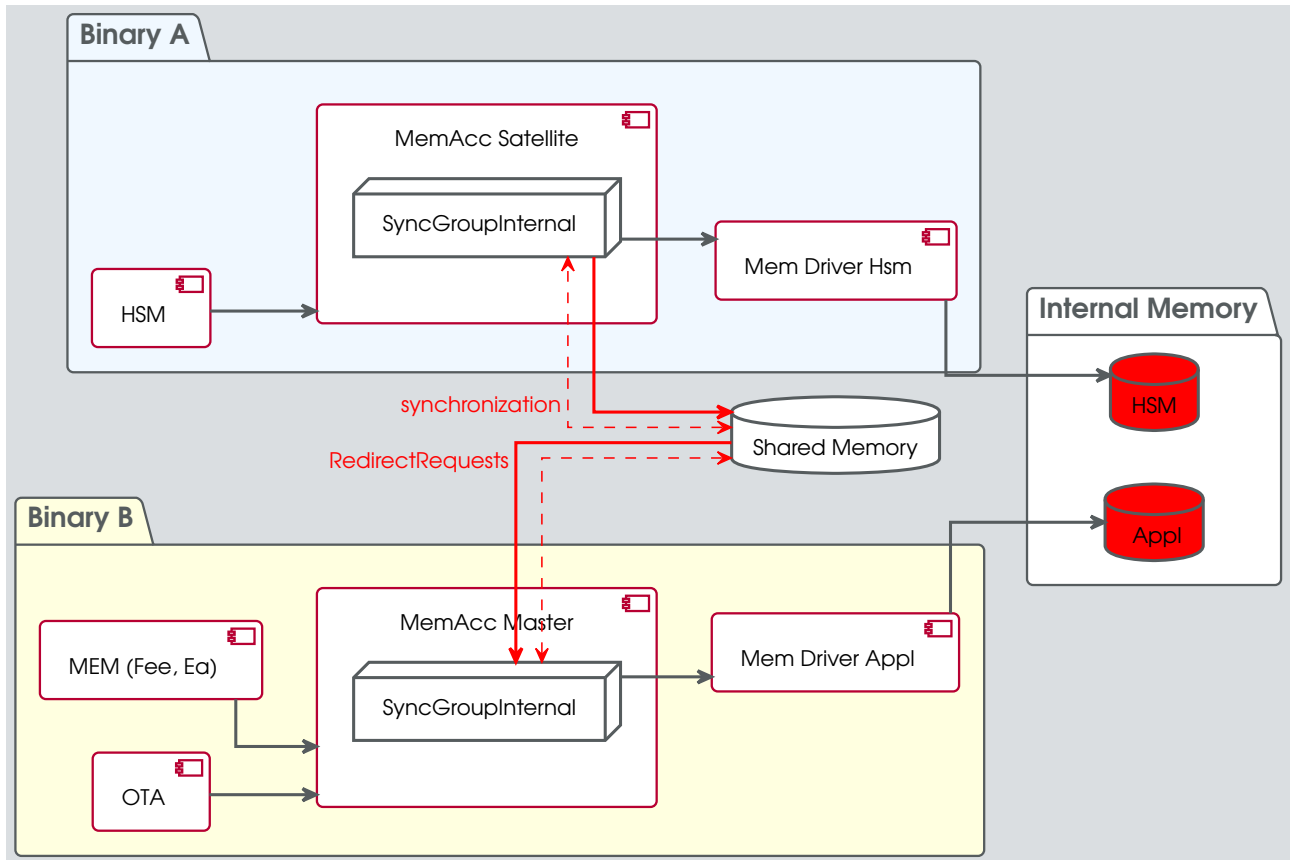


Figure 2.4 Use case 3

The three shown use cases can be extended by having more than one satellite as well as different combinations of synchronization group configurations. There is also the possibility to configure a Mem driver within the master which is only used by the satellite.

2.1.8 NAND-Flash Memory Support

The module supports software-based bad block management for targeted NAND-Flash Memory devices. It uses the skip bad block management method without lookup table persistency to provide the user with a continuous logical address space even if some physical blocks may be bad.



Note

Due to the reliance on hardware-specific information from the memory, this feature, which operates outside the scope of the AUTOSAR Specification, can only be utilized using specific Vector-approved NAND-Mem Drivers.



Note

Since the skip bad block management method is implemented to work per sub address area, multiple sub address areas may not target the same physical memory in case of NAND devices.



Caution

The NAND-Flash Memory Support feature is incompatible with the Multi Binary Feature. The bad block information is not synchronized across binaries. Only memory (sub address areas) not targeting NAND-Flash are allowed for Multi Binary synchronization and job redirection.

Key Concepts:

1. Initial bad block detection:

- > To detect which NAND-Flash blocks are bad, all blocks are scanned and any detected bad blocks are added to an internal lookup table.
- > The initial block scanning is done for each NAND-Flash device after module initialization during `MemAcc_MainFunction()` calls. Depending on the NAND-Flash memory size and block size this can take some time and multiple `MemAcc_MainFunction()` calls.
- > Each NAND-Flash-Block takes at least one `MemAcc_MainFunction()` depending on the cycle time and the speed of the Mem.
- > NAND-Flash-Blocks that are located in separate address areas and synchronization groups are initialized in parallel which can improve the speed of the initial bad block detection.

- > During the scanning process, the module can still be used normally. User operations can run in parallel with the block scanning as long as they operate in different address areas and synchronization groups. If a user job targets the NAND-Flash, it will be executed after the initial bad block scan is complete for this address area and can therefore take longer than usual.
- > Users cannot detect whether the initial block scanning has finished.

2. Dynamic bad block detection and processing:

- > Write and Erase operations can detect new bad blocks during runtime.
- > Newly detected bad blocks are processed automatically. This may extend the time required for user jobs where bad blocks are detected and necessitates additional `MemAcc_MainFunction()` calls. Data within bad blocks are automatically recovered into a good block.

3. Further information on Bad block management:

- > The module skips known bad physical blocks during all user jobs (except `HwSpecific` services) to provide the user with a continuous logical address space per sub address area.
- > The skip method for bad block management is handled per sub address area. Users must not target multiple sub address areas in a single NAND-Flash job request.
- > Due to dynamic detection of bad blocks at runtime, writing and erasing information must be done sequentially within each sub address area.
- > Each sub address area that targets a NAND-Flash may contain bad blocks. As the logical and physical address space sizes are identical per sub address area, targeting high logical addresses may result in a `MEMACC_ERRORTYPE_BBM_TRANSLATEDADDRESSOUTOFBOUNDS` error. This happens since during logical to physical address translation, the physical address adds an offset of one block for each bad block prior to the targeted block in the sub address area. This is in accordance with the bad block management skip method.
- > NAND configuration is done by linking sub address areas to a Sector Batch of a NAND-Mem. For the NAND-Mem configuration please refer to the Mem technical reference. If a NAND-Flash Memory is used, the `MemAccAddressAreaConfiguration/MemAccErrorNotificationName` has to be configured for the associated address area.

**Caution**

According to industry standard there is a chance that about 2% of all blocks per Die are bad blocks [6]. If the sub address area includes both the Dies in the device, please ensure at least 4% of blocks are reserved for bad block management. If individual sub address areas are created for individual Dies, each sub address area should reserve at least 2%. Please note that the percentages for the reserved blocks per sub address area are lower bounds and can be increased further to stay on the safe side. For more information please refer to the Mem technical reference. Reserved blocks of sub address areas should not be directly targeted by jobs of higher level components.

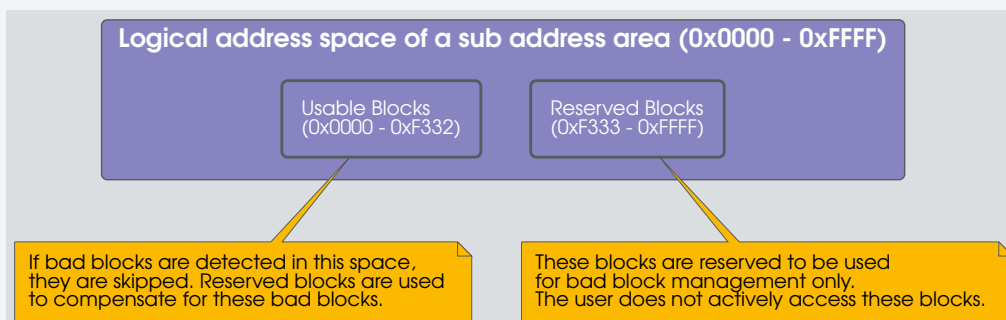


Figure 2.5 Used and reserved blocks in logical address space of a sub address area.

Runtime Error Reporting:

MemAcc provides a configurable error notification callback function for each address area. This function notifies the user about unhandled internal errors. It is in the responsibility of the user to log and handle the consequences (e.g. MEMACC_ERRORTYPE_BBM_BBLUT-FULL: Address Area is not usable anymore as all configured bad blocks are already used). If unhandled internal error happens during a user job the job will fail.

Error Type	Reason	Possible Impact
MEMACC_ERRORTYPE_BBM_READBBMARKER_REJECTED	Initial block scanning failed. Read bad block marker job was rejected.	Could be DET related. Check your configuration. Initialization failed.
MEMACC_ERRORTYPE_BBM_READBBMARKER_FAILED	Initial block scanning failed. Read bad block marker job failed.	Bad block information not readable. Initialization failed.
MEMACC_ERRORTYPE_BBM_RECOVERSECTOR_REJECTED	Bad block sector recovery was rejected by the Mem.	Could be DET related. Check your configuration.
MEMACC_ERRORTYPE_BBM_RECOVERSECTOR_FAILED	Recover Sector job failed.	Bad block recovery failed. Possible data loss during bad block processing.

continues on next page

Table 2.1 – continued from previous page

Error Type	Reason	Possible Impact
MEMACC_ERRORTYPE_BBM_ERASEBLOCK_REJECTED	Erase block job was rejected by the Mem.	Could be DET related. Check your configuration.
MEMACC_ERRORTYPE_BBM_ERASEBLOCK_FAILED	Erase block job failed.	Should never happen.
MEMACC_ERRORTYPE_BBM_WRITEBBMARKER_REJECTED	Write bad block marker job was rejected by the Mem.	Could be DET related. Check your configuration.
MEMACC_ERRORTYPE_BBM_WRITEBBMARKER_FAILED	Write bad block marker job failed.	Bad block information could not be persisted and is lost after ECU Reset. Correct address translation cannot be ensured.
MEMACC_ERRORTYPE_BBM_READERRORSTATE_REJECTED	Read Error State job was rejected by the Mem.	Could be DET related. Check your configuration.
MEMACC_ERRORTYPE_BBM_READERRORSTATE_INVALIDADDRESS	Read Error State job returned a invalid physical address.	Should never happen.
MEMACC_ERRORTYPE_BBM_BBLUTFULL	The maximum amount of bad blocks for a MemInstance has been reached.	No new bad block can be handled.
MEMACC_ERRORTYPE_BBM_RECOVERYTARGETOUTOFBOUNDS	The bad block recovery target is out of bounds of the current subaddress area.	No recovery is possible since no more sectors for recovery are available.
MEMACC_ERRORTYPE_BBM_TRANSLATEDADDRESSOUTOFBOUNDS	The calculated physical address is out of bounds of the current subaddress area. This can happen when the user access high addresses of a subaddress area when bad blocks exist.	Invalid physical address. Job cannot be dispatched to Mem.
MEMACC_ERRORTYPE_BBM_UNEXPECTED_P_FAIL	Mem reported an unexpected P_FAIL error.	Should never happen.
MEMACC_ERRORTYPE_BBM_UNEXPECTED_E_FAIL	Mem reported an unexpected E_FAIL error.	Should never happen.

2.1.9 Read only mode for specific memory areas

The MemAcc provides the possibility to set specific configured memory areas in a read only mode.

Definition Read Only Mode: Read Only mode means only services are usable which do not modify the memory content. Specifically this means all services are usable except `MemAcc_Write`, `MemAcc_Erase`. Full Mode means on the other side that all services are usable. As for `MemAcc_HwSpecificService` it depends on the specific implementation of the underlying Mem driver (if write/erase operations are done). Therefore, it is configurable whether this service is also usable in read only mode or not.

By default after initialization (`MemAcc_Init` and `MemAcc_DeInit`) the configured memory area is in read only mode. If a memory area is in read only mode, a request for a write and erase (if configured also `HwSpecificService`) job will be rejected or fails.

Use Cases: An example use case is if write and erase operations need to be restricted for program flash and shall only be enabled for a specific time window (e.g. software update). In this case, during normal operation the program flash memory area is set to read only mode to avoid accidental modification of the program code. Before a software update operation the memory area can be switched to full mode to enable write and erase operations. After the software update operation the memory area can be switched back to read only mode.

Normally, this requirement is fulfilled by using a Mem driver for program flash which will be dynamically activated via `MemAcc_ActivateMem` and `MemAcc_DeactivateMem`. But if this is not possible due to several reasons, e.g.

- > Mem driver does not provide dynamic activation, like Mems for external flash devices
- > Mem not fulfilling requirements for dynamic activation, like read only variant of the Mem shall be available and access to data flash shall be always possible within an own Mem driver

this feature can be used as an alternative.

Besides of this bootloader use case, also other use cases are possible.

Handling: The AUTOSAR services `MemAcc_ActivateMem` and `MemAcc_DeactivateMem` are re-used to switch from read only to full mode and back. As specified by AUTOSAR, both services will dynamically activate and deactivate a whole `INDIRECT_DYNAMIC` Mem driver (for more information see chapter *INDIRECT_DYNAMIC Mems and their dynamic activation*). With the extended read only mode feature both services can also be used for `DIRECT_STATIC` and `INDIRECT_STATIC` Mem drivers to switch the full mode on and off. To enable this feature for a specific memory area the optional container `/MICROSAR/MemAcc/MemAccReadOnlyModeConfiguration` needs to be configured:

- > With the references `MemAccAddressAreaLockRef` and `MemAccMemInstanceLockRef` a specific area can be chosen which shall be locked. E.g. if a MemInstance provides access to program as well as data flash, specifically the program flash area can be locked by choosing the AddressArea which manages the program flash.
- > With the parameter `MemAccUnlockTokenValue` a token value is configured which must be forwarded by the user to unlock the memory area. The token value is a string representing a byte array of variable length, e.g. it can be configured as `0xAABBCCDD` or `0x11` and so on.
- > With the parameter `MemAccReadOnlyModeSupportsHwSpecificService` it is configurable if the hardware specific service is supported in read only mode or not.

Read only mode container can be configured several times for different memory areas.

The AUTOSAR services `MemAcc_ActivateMem` and `MemAcc_DeactivateMem` both have

- > `hwld` as parameter. For the extended read only mode feature the `hwld` of the configured `MemAccMemInstanceLockRef` should be used.
- > `headerAddress` as parameter. For the extended read only mode feature the `headerAddress` points to the token value. E.g. if `MemAccUnlockTokenValue` is configured as `0x8899AABB` the MemAcc expects, beginning from the `headerAddress`, this exact byte

sequence. Only if this is the case the mode will be switched, otherwise `E_NOT_OK` will be returned.

2.2 Error Handling

2.2.1 Development Error Reporting

The module supports development error detection and reporting using the AUTOSAR Det. If enabled, these errors are reported using the service `Det_ReportError()` as specified in [3],

The reported service IDs and error IDs are contained and documented in the module header file.

2.3 Compliance

These are the major **unsupported** features compared to AUTOSAR [1]

> Support Post build

The MemAcc is implemented against AUTOSAR 23-11 with the following additional extensions and fixes:

- > For performance optimization *Parallel Processing of Different Asynchronous Jobs* is supported.
- > MemAcc supports synchronization and job redirection over multi binaries.
- > MemAcc supports bad block management of NAND devices (skip method of bad blocks)
- > MemAcc supports read only mode for specific memory areas.
- > It is planned for the next AUTOSAR versions to move the `MemAccMemInvocation` and `MemAccMemNamePrefix` parameters, which are currently located in the `MemAccSubAddressAreaConfiguration` container, to the Mem driver `MemGeneral` container. The MemAcc already detects Mem drivers that provide these parameters and will automatically synchronize these values up to the MemAcc to ease integration.
- > AUTOSAR defines besides the standard return values `E_OK` and `E_NOT_OK` also `E_MEM_SERVICE_NOT_AVAIL`. The return value `E_MEM_SERVICE_NOT_AVAIL` will not be used by the MemAcc as concrete handling is not clearly defined. Currently, if a Mem driver service returns for a request `E_MEM_SERVICE_NOT_AVAIL` the job result of the MemAcc will be `MEMACC_FAILED`.
- > The blank check job will be split into `MemWritePageSize` steps instead of the specified `MemMinReadSize`.
- > The missing function pointer for the Mem `DeInit()` function was added to the `MemAcc_MemBinaryHeaderType`. Also, wrong specified function pointer were corrected to match the signature of the Mem services specified within the Specification of Memory Driver. Also, header and Delimiter type were changed from `uint64` to `MemAcc_AddressType` to be compliant to the Mem AUTOSAR specification (see also [1] and [2]).

- > `MemAcc_RequestLock` locks only the respective Mem instances rather than the whole Mem drivers, see *MemAcc_RequestLock*.

3 Integration

3.1 Embedded Implementation

The component contains the following source code files. Some files are generated by the configuration tool DaVinci Configurator Classic.

File Name	Description	Integration Tasks
MemAcc.c	This is the source file of the module. It contains the implementation of the module interfaces.	-
MemAcc.h	This is the header file of the module. It declares the interfaces.	-
MemAcc_GeneralTypes.h	This file defines general types used by this component.	-
MemAcc_MemApi.h	This file defines the interfaces between the MemAcc and the Mem drivers. This file contains the MemBinaryHeaderType containing the function pointers of all Mem driver services, which must be provided by the Mem drivers if these are configured in INDIRECT_STATIC or INDIRECT_DYNAMIC mode.	-
MemAcc_Callout.h	This file defines callout functions that are expected to be implemented by the user.	-
MemAcc_Cfg.h	This file contains the public configuration parameters of the module.	This file is generated.
MemAcc_Lcfg.c	This file contains configuration parameters.	This file is generated.
MemAcc_MemCfg.h	This file contains the MemAcc_HwIdType.	This file is generated.

Table 3.1 Source Code Files

3.2 Initialization

3.2.1 Single Binary Use Case

Before calling any functionality, the module has to be initialized by calling `MemAcc_Init()`. The internal variable structures only need to be initialized via the service `MemAcc_InitMemory()`, in case they are not initialized by the startup code.

The **EcuM** usually takes care of initialization. If no EcuM is used these functions have to be called by the application in the correct order.

3.2.2 Multi Binary Use Case

Before calling any functionality of a master MemAcc the module has to be initialized by calling `MemAcc_Init()`. Also before calling any functionality of the satellites MemAcc(s), the corresponding module(s) have to be initialized by calling their `MemAcc_Init()`. The integration and handling of the shared memory variables needs special considerations. For more information refer to chapter Multi Binary (*Multi Binary*).

The shared memory variables are all defined by the MemAcc master binary and are also directly initialized as C variables. These variables are therefore placed in an INIT type of memory section (see chapter *Memory Sections for Multi Binary use case*), and it is necessary for the integration that after power up or reset these variables are initialized during startup code.

**Caution**

Re-initialization (`MemAcc_Init()` / `MemAcc_DeInit()`) of only one part (e.g. master or satellite) is not allowed as in this scenario the counter part (e.g. satellite or master) can be stucked or synchronization will not work anymore.

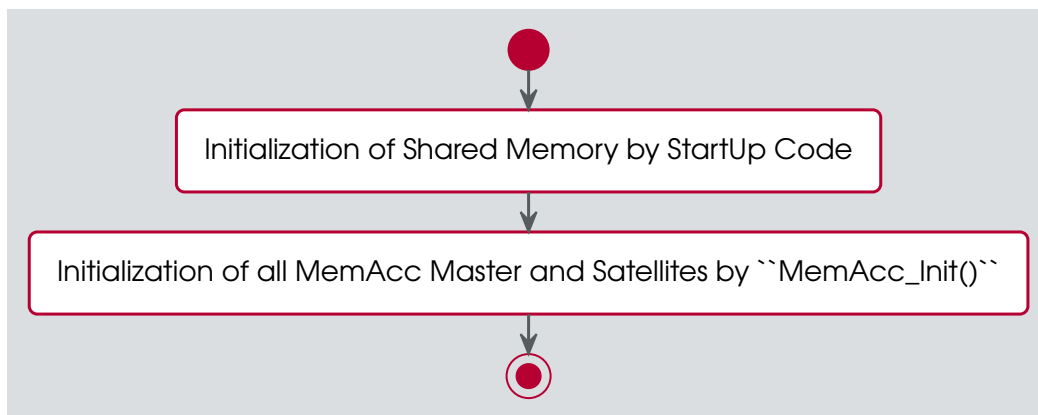


Figure 3.1 Overview: Initialization in the multi binary use case

3.3 Main Functions

3.3.1 Single Binary Use Case

The module has one main function:

```
MemAcc_MainFunction().
```

This main function has to be called with a

- > fixed cycle time, or
- > within a background task.

The cycle time can be configured in the configuration tool. Before calling the `MemAcc_MainFunction()` the module has to be initialized.

3.3.2 Multi Binary Use Case

Each binary (MemAcc module) has its own main function `MemAcc_MainFunction()` which has to be triggered. Before the `MemAcc_MainFunction()` of a binary is called, the module must be initialized.

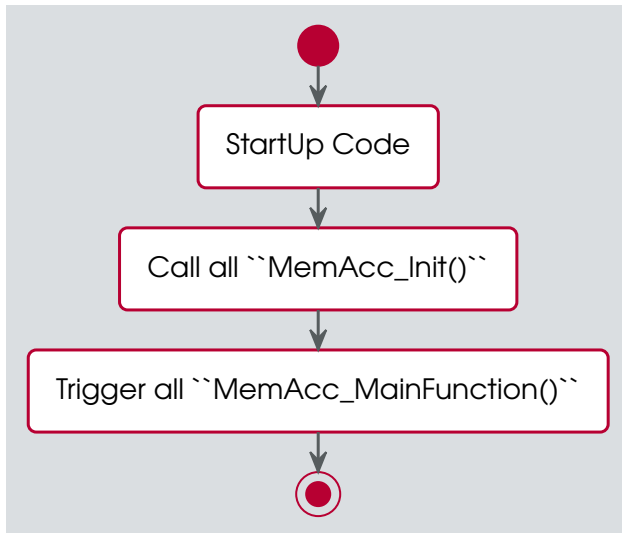


Figure 3.2 Overview: Triggering in the multi binary use case

In case the multi binary feature is used the following scenarios are thinkable:

Which MemAcc Binary is triggered?	Which jobs are processed?
only master	All master jobs can be processed.
master and a subset of satellites	All master jobs as well as the jobs from subset of satellites can be processed.
master and all satellites	All master jobs as well as all satellites job can be processed.
only satellite	The satellite can only process jobs for synchronization groups NOT relevant for multi binary synchronization.

3.4 Synchronization

3.4.1 Critical Sections

To protect its internal data structures against unwanted modification, the component uses critical sections, which prevent concurrent access. This module uses an exclusive area:

> `MEMACC_EXCLUSIVE_AREA_1`

for example.

The exclusive areas cover only one or a few statements and have a short runtime. The critical sections are used internally and they do not cover calls to other modules. The MemAcc does not nest critical sections.

In preemptive systems, it is necessary to protect some actions of preemption to avoid data mismatch. That means that a few MemAcc internal allocations need to be atomic. Therefore an exclusive area is used to ensure an atomic access, for example `MEMACC_EXCLUSIVE_AREA_1`.

One possible scenario for which atomic access is guaranteed:

If the MemAcc user is running on a different OS task from MemAcc itself and if either task is "preemptive", then access to uint32 variables must be protected unless it is inherently atomic.



Note

The exclusive area is only needed, if access to uint32 variables is not atomic and if the API `MemAcc_GetProcessedLength` is used.

3.5 Memory Sections

General information about memory mapping can be found within [4] and [5].



Caution

The user shall not map `MemAcc_CompareBuffer` (if compare is enabled via `MemAccGeneral/MemAccCompareApi`) into the cached RAM as the `MemAcc_CompareBuffer` can be accessed by hardware as well as the MemAcc. This also applies to variables of the shared memory section (see chapter *Memory Sections for Multi Binary use case*), which are accessed by multi binaries.

3.5.1 Multi Binary use case

The multi binary synchronization variables declared by MemAcc master binary must be mapped into an appropriate shared RAM region accessible by the master as well as all satellites.

The name of the shared memory section is:

```
MEMACC_START_SEC_VAR_NOCACHE_INIT_UNSPECIFIED
MEMACC_STOP_SEC_VAR_NOCACHE_INIT_UNSPECIFIED
```

The affected data, which are mapped into this shared memory section, are listed below:

- > `MemAcc_MultiBinary_SynchronizationToken`
- > `MemAcc_MultiBinary_AccessRequestSatellite_Id<SatellitIdNr>`
The following data are only relevant in shared memory for satellites which actually redirect requests.
- > `MemAcc_MultiBinary_Satellite_Data_Id<SatellitIdNr>`
- > `MemAcc_MultiBinary_RedirectJobStepInfo_Id<SatellitIdNr>`
- > `MemAcc_MultiBinary_RedirectJobStepResult_Id<SatellitIdNr>`

**Note**

This memory section can be ignored for the default single binary use case and if the MemAcc is configured as satellite.

3.6 Linker Symbols for Multi Binary use case

In case of multi binary use case, the linker symbol defines the address where the synchronization-related data are in shared RAM. The MemAcc as the master binary defines and declares these variables. The satellite binaries do not declare any variables. In satellites the data is accessed via extern declarations of the given linker symbols. The user needs to create the linker symbols pointing to the variables defined by the master.

The naming of linker symbols:

- > `MemAcc_MultiBinary_SynchronizationToken` (pointing to the `MemAcc_MultiBinary_SynchronizationToken` variable provided by the master MemAcc)
- > `MemAcc_MultiBinary_AccessRequestSatellite_Id<SatelliteId>` (pointing to the appropriate `MemAcc_MultiBinary_AccessRequestSatellite_Id<SatelliteId>` variable provided by the master MemAcc)

The `SatelliteId` itself is configured for each satellite within its configuration, starting with one.

**Note**

The names of the linker symbols for each satellite are fix and are also displayed in the satellite configurations. The linker symbols for each MemAcc satellite must be created and must be set to the appropriate address by the integrator.

3.7 Callouts

The component has callout functions. A user may provide an implementation for them based on their API description.

Prototype

```
void MemAcc_Callout_DataMemoryBarrier()
```

Functional Description

The User's implementation of the callout is invoked when a data memory barrier is requested. This memory barrier must ensure that all write accesses preceding the barrier are completed before any write accesses following the barrier are executed.

This callout is required only if the multi binary use case is enabled. If the multi binary use case is enabled, the memory barrier is only needed if compiler and/or CPU optimization is enabled and out-of-order execution is possible. If out-of-order execution is not possible this function can be implemented as a empty function.

Options

continues on next page

Table 3.3 – continued from previous page

callcontext: TASK reentrant: FALSE synchronous: TRUE
--

Table 3.3 MemAcc_Callout_DataMemoryBarrier

3.8 Error Reporting Det

The component has these properties used for error reporting to the Det component.

- > MEMACC_MODULE_ID = 41
- > MEMACC_VENDOR_ID = 30

3.9 INDIRECT_STATIC Mems

If the MemAcc references within the configuration directly a Mem driver with invocation type `INDIRECT_STATIC` a function pointer table must be provided by this Mem. The function pointer table must be of type `MemAcc_MemBinaryHeaderType` provided by `MemAcc_MemApi.h`.

For `INDIRECT_STATIC` Mems, the function pointer table has to be defined by the Mem using following naming scheme: `Mem_[Prefix]_FunctionPointerTable`. MemAcc uses a extern declaration to access this table and uses its function pointer to forward requests to the Mem. If the configured `INDIRECT_STATIC` Mem does not provide a `Mem_[Prefix]_FunctionPointerTable` of type `MemAcc_MemBinaryHeaderType` it should be provided by the integrator.

3.10 INDIRECT_DYNAMIC Mems and their dynamic activation

An `INDIRECT_DYNAMIC` Mem needs to be dynamically activated during runtime by calling `MemAcc_ActivateMem` for this specific Mem. If a Mem configured as `INDIRECT_DYNAMIC` is not activated, a job for this Mem will be rejected (if first job step targets `INDIRECT_DYNAMIC` Mem) or will fail with `MEMACC_FAILED` (if all following job steps target an `INDIRECT_DYNAMIC` Mem). If a dynamic Mem is not needed anymore it can be deactivated via `MemAcc_DeactivateMem`.

The dynamic Mem driver should be compiled and linked as a separate binary and must be loaded by the user into a predefined RAM area to be executed. The header address which is also part of the Mem binary header (see also [2]) must be static. Relocatable Binaries as specified by AUTOSAR are not supported.

**Note**

A Mem configured as `INDIRECT_DYNAMIC` can be activated and deactivated several times. For example if first only read access should be allowed then an image of a Mem can be activated which only provides read functionality. If later also write/erase access should be allowed, the Mem can then be deactivated again and an image of a Mem can be activated which provides all functionality. In this case it is important that both Mem images are based on MemS with the same `MemSectorBatch` configuration.

**Note**

If a Mem is configured with more than one instance, forwarding a hwld representing one specific instance of this Mem within `MemAcc_ActivateMem`, `MemAcc_DeactivateMem` will activate/deactivate the whole Mem driver with all instances. This behaviour is based on the fact that one Mem driver provide one binary header for all instances and the Mem driver itself can only be initialized and deinitialized as a whole. E.g., if a Mem driver exist with one instance for data flash and one instance for program flash access and only the program flash access shall be activated dynamically, two versions of this Mem driver must be configured and provided:

- > The Mem driver for data flash access configured as `DIRECT/INDIRECT_STATIC` and the Mem driver for program flash access configured as `INDIRECT_DYNAMIC` which will be compiled and linked into an separate image.

Example configuration in the following figure 3.3:

`Mem_PFlashAccess` configured as `INDIRECT_DYNAMIC` which needs to be compiled and linked as a separate image. If there are several variants of this Mem, e.g. one with read only and one with full memory access, all MemS must have the same `MemSectorBatch_DFlash` configuration as the original configured `INDIRECT_DYNAMIC` Mem referenced by `MemAcc`.

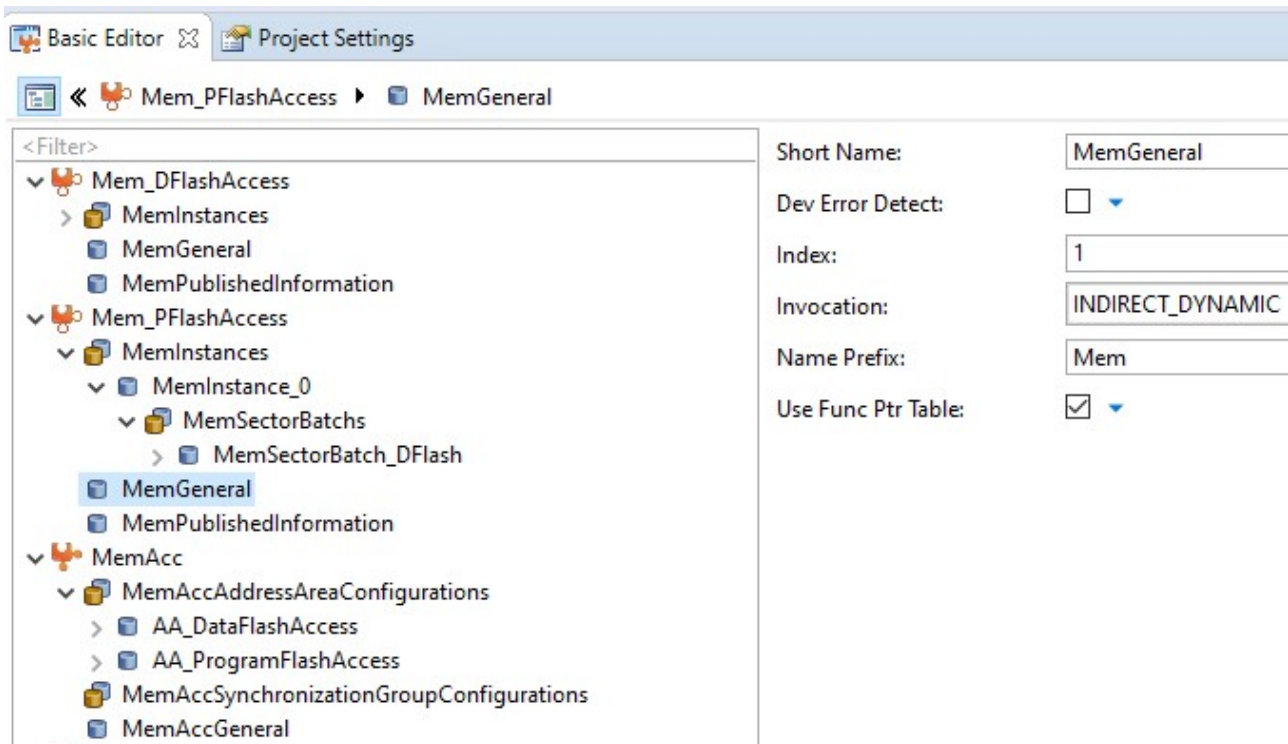


Figure 3.3 INDIRECT_DYNAMIC example configuration

Also Vector specific vMem drivers are usable for dynamic activation/deactivation via the Mem_30_LegacyAdapter. These vMem drivers also provide a configurable read only mode (see also [7]). When creating the image of the Mem which shall be dynamically activated it is important to ensure that the header of the Mem_30_Legacy is at the start of the image and the delimiter of the Mem_30_Legacy is in the end.

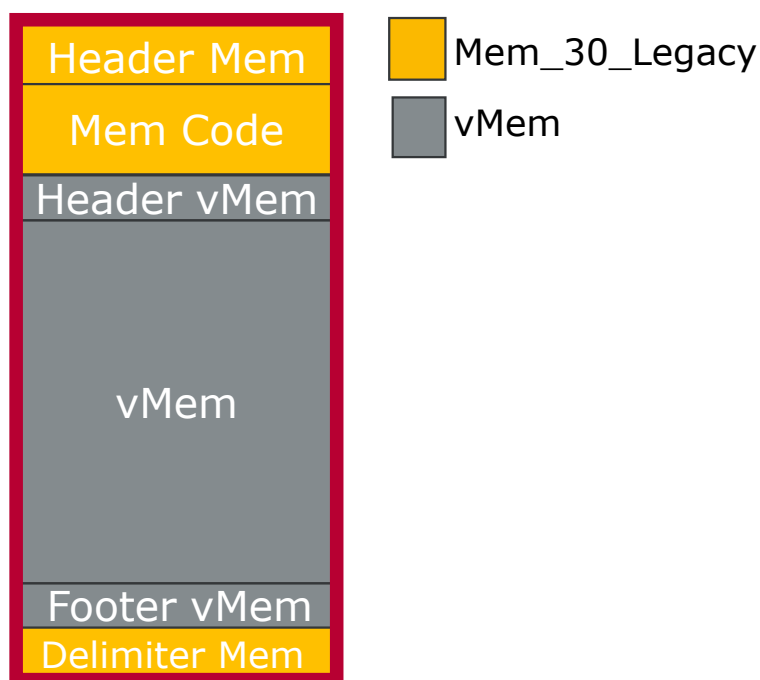


Figure 3.4 Binary image for legacy vMem activation

Typically the dynamic activation feature is used when access to flash shall be only enabled for a specific time. As an alternative for the dynamic activation of a whole Mem driver, the MemAcc provides the possibility to set specific memory areas into read only mode. For more information see chapter *Read only mode for specific memory areas*.

3.11 Handling of ECC Errors

Depending on the platform it might be necessary for the integration to implement ECC interrupts/exceptions to handle correctable or uncorrectable ECC errors. The provided `MemAcc_GetJobInfo()` service can be used within the ECC exception handler to determine if the ECC error was triggered by a Mem driver. If this is the case, the error itself can be propagated to the specific Mem driver by using the respective `Mem_[Prefix]_PropagateError()` callback. For more information see [2] and the technical references of the underlying Mem drivers.

3.12 Compatibility with former AUTOSAR releases

Between AUTOSAR 23-11 and the previous AUTOSAR release 22-11 there were minor changes regarding the interfaces provided by the MemAcc as well as the underlying Mem driver:

- > The naming of `MemAcc_JobResultType` enumeration members and `MemAcc_MemoryInfoType` structure members changed slightly (see [1]). If all users of the MemAcc are expecting a 22-11 MemAcc the integrator should supply the following define to the C preprocessor through the build environment: `MEMACC_ASR22_11_COMPATIBILITY`.
- > The naming of Mem driver parameter `MemReadPageSize` changed to `MemMinReadSize` and `MemReadBurstSize` changed to `MemMaxReadSize` (see [2]). The MemAcc referencing Mem drivers based on AUTOSAR 22-11 is by default possible. `MemReadPageSize` will be automatically interpreted as `MemMinReadSize` and `MemReadBurstSize` as `MemMaxReadSize`. In this case, the Mem driver must accept any read request length between `MemMinReadSize` and `MemMaxReadSize` (aligned to `MemMinReadSize`).

If a user of the MemAcc also accesses Mem read configuration information directly this could lead to an issue if the Mem driver configuration is not according to the AUTOSAR version the user expected. AUTOSAR 22-11 parameter `MemAccUseReadBurst` is not defined by the MemAcc anymore as this parameter was never meaningful for read request handling.

4 API Description

4.1 Type Definitions

The types defined by the component are declared in `MemAcc_GeneralTypes.h`.

4.2 Provided Functions

The component provides the following functions.

Service functions are implemented by the module and are typically called by upper-layer modules. They are declared in `*.h`.



Note

Since only one memory job is allowed per address area, it is a 1:1 relation between address area and upper-layer modules (user).

4.2.1 MemAcc_InitMemory

Prototype	
void MemAcc_InitMemory ()	
Functional Description	
Initializes MemAcc variables Service to initialize module global variables at power up.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.1 MemAcc_InitMemory

4.2.2 MemAcc_Init

Prototype	
void MemAcc_Init (const MemAcc_ConfigType* configPtr)	
Parameters	
configPtr [in]	Pointer to the MemAcc config (This parameter is currently not used.)
Functional Description	
Initialization function of MemAcc All variables are set to default values. The module state is set to initialized.	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

continues on next page

Table 4.2 – continued from previous page

Table 4.2 MemAcc_Init

4.2.3 MemAcc_DeInit

Prototype
void MemAcc_DeInit ()
Functional Description
Deinitialization function of MemAcc The service deinitialize the MemAcc modules internal states.
Options
callcontext: TASK reentrant: FALSE synchronous: TRUE

Table 4.3 MemAcc_DeInit

4.2.4 MemAcc_GetVersionInfo

Prototype	
void MemAcc_GetVersionInfo (Std_VersionInfoType* versionInfoPtr)	
Parameters	
versionInfoPtr [in]	Pointer to where to store the version information of this module.
Functional Description	
Retrieves the version information of the MemAcc module.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.4 MemAcc_GetVersionInfo

4.2.5 MemAcc_GetJobResult

Prototype	
MemAcc_JobResultType MemAcc_GetJobResult (MemAcc_AddressArealdType addressAreald)	
Parameters	
addressAreald [in]	Numeric identifier of address area
Return Codes	
MEMACC_OK: The last job was finished successfully.	
MEMACC_FAILED: The last job resulted in unspecific failure, the job was not completed.	
MEMACC_INCONSISTENT: The result of the last job did not meet the expected result.	
MEMACC_CANCELED: The last MemAcc job was canceled.	
MEMACC_ECC_UNCORRECTED: The last memory operation returned an uncorrectable ECC error.	
MEMACC_ECC_CORRECTED: The last memory operation returned a correctable ECC error.	

continues on next page

Table 4.5 – continued from previous page

Functional Description
This function returns the consolidated job result of the address area referenced by addressAreald.
Options
callcontext: TASK reentrant: TRUE synchronous: TRUE

Table 4.5 MemAcc_GetJobResult

4.2.6 MemAcc_GetJobStatus

Prototype	
MemAcc_JobStatusType MemAcc_GetJobStatus (MemAcc_AddressArealdType addressAreald)	
Parameters	
addressAreald [in]	Numeric identifier of address area
Return Codes	
MEMACC_JOB_IDLE: MemAcc is not processing a job for the requested addressAreald.	
MEMACC_JOB_PENDING: MemAcc is currently processing a job for the requested addressAreald.	
Functional Description	
This function returns the status of the MemAcc job referenced by addressAreald.	
Options	
callcontext: TASK	
reentrant: TRUE	
synchronous: TRUE	

Table 4.6 MemAcc_GetJobStatus

4.2.7 MemAcc_GetMemoryInfo

Prototype	
Std_ReturnType MemAcc_GetMemoryInfo (MemAcc_AddressArealdType addressAreald, MemAcc_AddressType address, MemAcc_MemoryInfoType* memoryInfoPtr)	
Parameters	
addressAreald [in]	Numeric identifier of address area
address [in]	Address in logical address space from which corresponding memory device information shall be retrieved.
memoryInfoPtr [in]	Destination memory pointer to store the memory device information.
Return Codes	
E_OK: The requested addressAreald and address are valid.	
E_NOT_OK: The requested addressAreald and address are invalid.	
Functional Description	
This service function retrieves the physical memory device information of a specific address area.	
Options	

continues on next page

Table 4.7 – continued from previous page

callcontext: TASK reentrant: TRUE synchronous: TRUE
Table 4.7 MemAcc_GetMemoryInfo

4.2.8 MemAcc_GetProcessedLength

Prototype	
MemAcc_LengthType MemAcc_GetProcessedLength (MemAcc_AddressArealdType addressAreald)	
Parameters	
addressAreald [in]	Numeric identifier of address area
Return Codes	
Processed length of current job (in bytes). If the requested addressAreald is invalid: 0u	
Functional Description	
This function returns the accumulated number of bytes that have already been processed in the current job.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.8 MemAcc_GetProcessedLength

4.2.9 MemAcc_GetJobInfo

Prototype	
void MemAcc_GetJobInfo (MemAcc_AddressArealdType addressAreald, MemAcc_JobInfoType* jobInfoPtr)	
Parameters	
addressAreald [in]	Numeric identifier of address area
jobInfoPtr [in]	Structure pointer to return the detailed processing information of the current job.
Functional Description	

continues on next page

Table 4.9 – continued from previous page

This service function retrieves detailed information about the current memory job like memory driver hardware identifier, job type, job processing state or job result, as well as address and length. The service distinguishes between invalid function calls and calls where no job is queued, a job is queued, and a job is running:

- > **Invalid parameters:** jobInfoPtr remains unmodified.
- > **No job queued:** jobInfoPtr returns the information of the last job. In this case CurrentJob=NO_JOB.
- > **Job queued, not running:** distinguishes the two subcases:
 - > hardware specific service jobs: LogicalAddress=0, Length=0, HwId=<value>, MemInstancId=0, MemAddress=0, MemLength=0, CurrentJob=<value>, and MemResult=<value> (of previous job).
 - > all other jobs: LogicalAddress=<value>, Length=<value>, HwId=0, MemInstancId=0, MemAddress=0, MemLength=0, CurrentJob=<value>, and MemResult=<value> (of previous job).
- > **Job queued and running:** distinguishes the two subcases:
 - > hardware specific service jobs: LogicalAddress=0, Length=0, HwId=<value>, MemInstancId=<value>, MemAddress=0, MemLength=0, CurrentJob=<value>, and MemResult=MEM_JOB_PENDING.
 - > all other jobs: LogicalAddress=<value>, Length=<value>, HwId=<value>, MemInstancId=<value>, MemAddress=<value>, MemLength=<value>, CurrentJob=<value>, and MemResult=MEM_JOB_PENDING.

Options

callcontext: TASK
reentrant: TRUE
synchronous: TRUE

Table 4.9 MemAcc_GetJobInfo

4.2.10 MemAcc_MainFunction

Prototype

void **MemAcc_MainFunction**()

Functional Description

This function handles the requested jobs and the internal management operations.

Options

callcontext: TASK
reentrant: FALSE
synchronous: TRUE

Table 4.10 MemAcc_MainFunction

4.2.11 MemAcc_Cancel

Prototype

void **MemAcc_Cancel**(MemAcc_AddressAreaIdType addressAreaId)

Parameters

addressAreaId [in]	Numeric identifier of address area
--------------------	------------------------------------

Functional Description

continues on next page

Table 4.11 – continued from previous page

Mark a job for a given address area to be canceled. Marked jobs will be canceled during the following MainFunction or, if being processed by Mem, upon job step completion.

Options

callcontext: TASK
reentrant: TRUE (conditionally, for different addressAreald)
synchronous: FALSE

Table 4.11 MemAcc_Cancel

4.2.12 MemAcc_Read

Prototype

Std_ReturnType **MemAcc_Read**(MemAcc_AddressArealdType addressAreald, MemAcc_AddressType sourceAddress, MemAcc_DataType* destinationDataPtr, MemAcc_LengthType length)

Parameters

addressAreald [in]	Numeric identifier of address area
sourceAddress [in]	Read address in logical address space (aligned to MemMinReadSize) If the alignment requirement is disabled via configuration, the address can be arbitrary.
destinationDataPtr [in]	Application pointer to buffer to read to
length [in]	Read length in bytes (aligned to MemMinReadSize) If the alignment requirement is disabled via configuration, the length can be arbitrary.

Return Codes

E_OK: The requested job has been accepted by the module
E_NOT_OK: The requested job has not been accepted by the module

Functional Description

This function triggers a read job to copy data from the source address into the referenced destination data buffer. It is possible to disable the alignment requirement for the read address and length by configuration, see *Unaligned Read and Compare Support*.

Options

callcontext: TASK
reentrant: TRUE (conditionally, for different addressAreald)
synchronous: FALSE

Table 4.12 MemAcc_Read

4.2.13 MemAcc_Write

Prototype

Std_ReturnType **MemAcc_Write**(MemAcc_AddressArealdType addressAreald, MemAcc_AddressType targetAddress, const MemAcc_DataType* sourceDataPtr, MemAcc_LengthType length)

Parameters

addressAreald [in]	Numeric identifier of address area
--------------------	------------------------------------

continues on next page

Table 4.13 – continued from previous page

targetAddress [in]	Write address in logical address space (aligned to write page size)
sourceDataPtr [in]	Source data pointer (aligned to MemAccBufferAlignmentValue)
length [in]	Write length in bytes (aligned to write page size)
Return Codes	
E_OK: The requested job has been accepted by the module	
E_NOT_OK: The requested job has not been accepted by the module	
Functional Description	
This function triggers a write job to store the passed data to the provided address area with a given address and length.	
Options	
callcontext: TASK	
reentrant: TRUE (conditionally, for different addressAreald)	
synchronous: FALSE	

Table 4.13 MemAcc_Write

4.2.14 MemAcc_Erase

Prototype	
Std_ReturnType MemAcc_Erase (MemAcc_AddressArealdType addressAreald, MemAcc_AddressType targetAddress, MemAcc_LengthType length)	
Parameters	
addressAreald [in]	Numeric identifier of address area
targetAddress [in]	Erase address in logical address space (aligned to sector size)
length [in]	Erase length in bytes (aligned to sector size)
Return Codes	
E_OK: The requested job has been accepted by the module	
E_NOT_OK: The requested job has not been accepted by the module	
Functional Description	
This function triggers an erase job of the given area defined by targetAddress and length.	
Options	
callcontext: TASK	
reentrant: TRUE (conditionally, for different addressAreald)	
synchronous: FALSE	

Table 4.14 MemAcc_Erase

4.2.15 MemAcc_Compare

Prototype	
Std_ReturnType MemAcc_Compare (MemAcc_AddressArealdType addressAreald, MemAcc_AddressType sourceAddress, const MemAcc_DataType* dataPtr, MemAcc_LengthType length)	

continues on next page

Table 4.15 – continued from previous page

Parameters	
addressAreaId [in]	Numeric identifier of address area
sourceAddress [in]	Compare address in logical address space (aligned to MemMinReadSize) If the alignment requirement is disabled via configuration, the address can be arbitrary.
dataPtr [in]	Pointer to user data which shall be compared to data in memory
length [in]	Compare length in bytes (aligned to MemMinReadSize) If the alignment requirement is disabled via configuration, the length can be arbitrary.
Return Codes	
E_OK: The requested job has been accepted by the module	
E_NOT_OK: The requested job has not been accepted by the module	
Functional Description	
This function triggers a job to compare the passed data to memory content defined by sourceAddress and length. The underlying read operation is automatically performed with a valid aligned length and compared up to the desired length. If the compare operation determines a mismatch, the job result is set to MEMACC_INCONSISTENT. Correctable ECC errors of the underlying read operation are handled like successful read operations and result in a MEMACC_OK result code. In case of an uncorrectable ECC error or a general error in the read operation, the job result is MEMACC_ECC_UNCORRECTED or MEMACC_FAILED, respectively. It is possible to disable the alignment requirement for the read address and length by configuration, see <i>Unaligned Read and Compare Support</i> .	
Options	
callcontext: TASK	
reentrant: TRUE (conditionally, for different addressAreaId)	
synchronous: FALSE	

Table 4.15 MemAcc_Compare

4.2.16 MemAcc_BlankCheck

Prototype	
Std_ReturnType MemAcc_BlankCheck (MemAcc_AddressAreaIdType addressAreaId, MemAcc_AddressType targetAddress, MemAcc_LengthType length)	
Parameters	
addressAreaId [in]	Numeric identifier of address area
targetAddress [in]	Blank check address in logical address space (aligned to write page size)
length [in]	Blank check length in bytes (aligned to write page size)
Return Codes	
E_OK: The requested job has been accepted by the module	
E_NOT_OK: The requested job has not been accepted by the module	
Functional Description	
This function checks if the passed address space is blank, i.e. erased and writeable.	

continues on next page

Table 4.16 – continued from previous page

Options
callcontext: TASK reentrant: TRUE (conditionally, for different addressAreaId) synchronous: FALSE

Table 4.16 MemAcc_BlankCheck

4.2.17 MemAcc_HwSpecificService

Prototype	
Std_ReturnType MemAcc_HwSpecificService (MemAcc_AddressAreaIdType addressAreaId, MemAcc_HwIdType hwId, MemAcc_MemHwServiceIdType hwServiceId, MemAcc_DataType* dataPtr, MemAcc_LengthType* lengthPtr)	
Parameters	
addressAreaId [in]	Numeric identifier of address area
hwId [in]	Unique numeric memory driver identifier
hwServiceId [in]	Hardware specific service request identifier for dispatching the request
dataPtr [in]	Data pointer pointing to the job buffer. Value can be NULL_PTR, if not needed. If dataPtr is used by the hardware specific service, the pointer must be valid until the job is completed
lengthPtr [in]	Size pointer of the data passed by dataPtr. Can be NULL_PTR if dataPtr is also NULL_PTR
Return Codes	
E_OK: The requested job has been accepted by the module E_NOT_OK: The requested job has not been accepted by the module	
Functional Description	
This function triggers a hardware specific job request referenced by hwServiceId. Service specific data can be passed/retrieved by dataPtr. The hwId is provided by generated MemAcc_MemCfg.h file and can also be obtained via the service MemAcc_GetMemoryInfo().	
Options	
callcontext: TASK reentrant: TRUE (conditionally, for different addressAreaId) synchronous: FALSE	

Table 4.17 MemAcc_HwSpecificService

4.2.18 MemAcc_RequestLock

Prototype	
Std_ReturnType MemAcc_RequestLock (MemAcc_AddressAreaIdType addressAreaId, MemAcc_AddressType address, MemAcc_LengthType length, void* lockNotificationFctPtr)	
Parameters	
addressAreaId [in]	Numeric identifier of address area

continues on next page

Table 4.18 – continued from previous page

address [in]	Logical start address of the address area to identify the Mem instances to be locked
length [in]	Length of the address area to identify the Mem instances to be locked
lockNotificationFctPtr [in]	Pointer to address area lock notification callback function
Return Codes	
E_OK: The requested job has been accepted by the module	
E_NOT_OK: The requested job has not been accepted by the module	
Functional Description	
This function triggers a job to lock the Mem instances referenced by addressAreaId, address and length. Once the lock job is processed, a lockNotification function is called and the MemAcc does not forward memory requests to the respective Mem instances until the lock is released by <i>MemAcc_ReleaseLock</i>.	
Caution Ensure that <code>MemAcc_RequestLock</code> is called with a direct function pointer as <code>lockNotificationFctPtr</code> parameter, not a pointer to a function pointer, i.e. it should be called with the symbol of the lock notification function rather than the address of the symbol. <code>lockNotificationFctPtr</code> will not be dereferenced before calling.	
Caution If the locking feature is used in combination with the multi binary feature, the application is responsible to lock each satellite and master individually.	
Options	
callcontext: TASK	
reentrant: TRUE (conditionally, for different addressAreaId)	
synchronous: FALSE	

Table 4.18 MemAcc_RequestLock

4.2.19 MemAcc_ReleaseLock

Prototype	
Std_ReturnType MemAcc_ReleaseLock (MemAcc_AddressAreaIdType addressAreaId, MemAcc_AddressType address, MemAcc_LengthType length)	
Parameters	
addressAreaId [in]	Numeric identifier of address area
address [in]	Logical start address of the address area to identify the Mem instances to be unlocked

continues on next page

Table 4.19 – continued from previous page

length [in]	Length of the address area to identify the Mem instances to be unlocked
Return Codes	
E_OK: The requested job has been accepted by the module	
E_NOT_OK: The requested job has not been accepted by the module	
Functional Description	
This function triggers a job to release the lock on Mem instances referenced by addressAreald, address and length.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.19 MemAcc_ReleaseLock

4.2.20 MemAcc_ActivateMem

Prototype	
Std_ReturnType MemAcc_ActivateMem (MemAcc_AddressType headerAddress, MemAcc_HwldType hwld)	
Parameters	
headerAddress [in]	Physical start address of an INDIRECT_DYNAMIC Mem driver header structure (<i>AUTOSAR</i>) or a token value (<i>Extension</i>)
hwld [in]	Unique numeric memory driver identifier
Return Codes	
E_OK: Mem driver activation successfull (<i>AUTOSAR</i>) or Memory area unlocking from read only mode to full mode successful (<i>Extension</i>)	
E_NOT_OK: Mem driver activation failed (<i>AUTOSAR</i>) or Memory area unlocking failed (<i>Extension</i>)	
Functional Description	
As specified by <i>AUTOSAR</i> , this service dynamically activates and initializes an INDIRECT_DYNAMIC Mem driver referenced by hwld and headerAddress. As an <i>Extension</i> to the <i>AUTOSAR</i> functionality, with this service also configured memory areas of DIRECT_STATIC and INDIRECT_STATIC Mem driver can be unlocked from a read only mode to a full (write/erase) mode. For more information see chapter <i>Read only mode for specific memory areas</i> .	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.20 MemAcc_ActivateMem

4.2.21 MemAcc_DeactivateMem

Prototype	
Std_ReturnType MemAcc_DeactivateMem (MemAcc_AddressType headerAddress, MemAcc_HwldType hwld)	

continues on next page

Table 4.21 – continued from previous page

Parameters	
headerAddress [in]	Physical start address of an INDIRECT_DYNAMIC Mem driver header structure (<i>AUTOSAR</i>) or a token value (<i>Extension</i>)
hwld [in]	Unique numeric memory driver identifier
Return Codes	
E_OK: Mem driver deactivation successful (<i>AUTOSAR</i>) or Memory area locking from full mode to read only mode successful (<i>Extension</i>)	
E_NOT_OK: Mem driver deactivation failed (<i>AUTOSAR</i>) or Memory area locking failed (<i>Extension</i>)	
Functional Description	
As specified by <i>AUTOSAR</i> , this service dynamically deinitializes and deactivates an INDIRECT_DYNAMIC Mem driver referenced by hwld and headerAddress. As an <i>Extension</i> to the <i>AUTOSAR</i> functionality, with this service also configured memory areas of DIRECT_STATIC and INDIRECT_STATIC Mem driver can be locked from a full (write/erase) mode to a read only mode. For more information see chapter <i>Read only mode for specific memory areas</i> .	
Options	
callcontext: TASK reentrant: FALSE synchronous: TRUE	

Table 4.21 MemAcc_DeactivateMem

4.3 Required Functions

The component requires the following functions from other components. Refer to the respective documentation for details.

Component	API	
Det	Det_ReportError	
Mem	Mem_<VendorId>_<VendorApilInfix>_Read Mem_<VendorId>_<VendorApilInfix>_Write Mem_<VendorId>_<VendorApilInfix>_Erase Mem_<VendorId>_<VendorApilInfix>_BlankCheck Mem_<VendorId>_<VendorApilInfix>_HwSpecificService Mem_<VendorId>_<VendorApilInfix>_GetJobResult Mem_<VendorId>_<VendorApilInfix>_Resume Mem_<VendorId>_<VendorApilInfix>_Suspend Mem_<VendorId>_<VendorApilInfix>_Init Mem_<VendorId>_<VendorApilInfix>_DeInit Mem_<VendorId>_<VendorApilInfix>_MainFunction	
VStdLib	VStdLib_MemCmp	VStdLib_MemCpy

Table 4.22 Required Functions

4.4 Configurable Interfaces

At its configurable interfaces, the module defines notifications that can be mapped to callback functions provided by other modules.

Prototype	
void <AddressAreaJobEndNotification>(MemAcc_AddressAreaIdType addressAreaId, MemAcc_JobResultType jobResult)	
Parameters	
addressAreaId [in]	Numeric identifier of the address area for which a job finished.
jobResult [in]	MEMACC_OK last MemAcc job was finished successfully MEMACC_FAILED last MemAcc job resulted in an unspecific failure and the job was not completed MEMACC_INCONSISTENT result of the last MemAcc job did not meet the expected result MEMACC_CANCELED last MemAcc job was canceled MEMACC_ECC_UNCORRECTED last memory operation returned an uncorrectable ECC error MEMACC_ECC_CORRECTED last memory operation returned a correctable ECC error
Functional Description	
Notification, which is called upon job completion of an address area. The MemAcc is ready to accept new jobs within this notification.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.23 <AddressAreaJobEndNotification>

Prototype	
void <AddressAreaJobErrorNotification>(MemAcc_AddressAreaIdType addressAreaId, MemAcc_ErrorType errorType)	
Parameters	
addressAreaId [in]	Numeric identifier of the address area for which a job finished.
errorType [in]	Type of error that occurred, see <i>Runtime Error Reporting</i> .
Functional Description	
Notification, which is called when an unhandled error occurs during job processing.	
Options	
callcontext: TASK reentrant: TRUE synchronous: TRUE	

Table 4.24 <AddressAreaJobErrorNotification>

4.5 Service Ports

This module does not provide service ports.

5 Configuration

The functionalities of the component are configured with DaVinci Configurator Classic.

For details on configuration options refer to the module-specific description that is shown in DaVinci Configurator Classic.

5.1 Configuration Variants

This component supports these configuration variants:

> VARIANT-PRE-COMPILE

The configuration classes of the parameters depend on the supported configuration variants. For their definitions please see the module-specific BSWMD ARXML file.

5.2 Configuration with DaVinci Configurator Classic

The configuration parameters are generally well described within the tool and the BSWMD ARXML file. However, there are some more complex dependencies between certain configuration parameters which are described in more detail in the following.

5.2.1 Memory Address Mapping

To achieve a correct **mapping from logical to physical addresses** the sub address areas must be configured according to the following instructions:

Physical Memory

- > The sector batch where the physical memory of the related sub address area is located must be referenced in the parameter `MemAccSectorBatchRef`. All memory device specific information, such as `MemWritePageSize`, `MemEraseSectorSize`, `MemMinReadSize` and the optional `MemWriteBurstSize`, `MemEraseBurstSize` and `MemMaxReadSize`, are to be configured within the Mem driver settings in the respective sector batch.
- > The physical memory of any sub address area must lie within the range of the referenced sector batch. The range of a sector batch is determined by its start and end address:
 - > The start address of the sector batch has to be configured in the Mem driver parameter `Start Address`.
 - > The end address of the sector batch can be computed using the Mem driver parameter `Number Of Sectors` and `Erase Sector Size` using the formula: $End\ Address = Start\ Address + Number\ Of\ Sectors \times Erase\ Sector\ Size - 1$
- > The same sector batch can be referenced from multiple sub address areas from different address areas. The physical memory of the sub address areas referencing the same sector batch must NOT overlap within this sector batch.

Logical Memory

- > The sub address areas must not overlap within the address area.

- > The Logical Start Address of a sub address area is configured in the parameter Logical Start Address. The Number Of Sectors of a sub address area determines how many sectors from the referenced sector batch are used. The Logical End Address of a sub address area can be calculated using the formula: *Logical End Address = Logical Start Address + Number Of Sectors x Erase Sector Size – 1*
- > The logical end address of a sub address area must not exceed the value 4294956295 or 0xFFFFFFFF in hexadecimal, since the type used within the implementation is uint32.

5.2.2 Parallel Processing Configuration

Configuration of synchronization groups must be done via the optional container /MICROSAR/MemAcc/MemAccSynchronizationGroupConfiguration. When configuring synchronization groups, at least two instances of the container MemAccSynchronizationGroupConfiguration must be created. Otherwise, by default, just one synchronization group exists, which all sub address areas will be part of.

The parameter MemAccSynchronizationSubAddressAreaRef holds references to sub address areas. When using synchronization groups, all sub address areas must be referenced by exactly one synchronization group instance. All sub address areas referenced by a synchronization group will be processed sequentially within that group.

5.2.3 Multi Binary Configuration

Configuration of the multi binary feature must be done via the optional container /MICROSAR/MemAcc/MemAccMultiBinaryConfiguration. In that case, precisely one binary, i.e. one configuration project must be configured as a master and the other binaries, at most 15, must be configured as satellites. Each binary receives a numeric identifier, where the master automatically is assigned the Id 0 and the satellites' Id must be configured to be at most the number of satellites as configured in the master (/MemAccMasterConfiguration/.../MemAccNumberOfSatellites) and must be pairwise different. All binaries must be uniformly configured to handle either a 32-bit or 64-bit address space (/MICROSAR/MemAcc/MemAccGeneral/MemAcc64BitSupport).

Further configuration for both master and satellite binaries is done within the container MemAccMultiBinarySynchronizationGroupConfiguration. If the parallel processing feature is used, that is if more than one synchronization group is configured, there must be one dedicated multi binary synchronization group. This relevant synchronization group must then be referenced in the parameter MemAccMultiBinarySynchronizationGroupConfiguration/MemAccRelevantSynchronizationGroup. Note, that if no synchronization group is referenced, by default all configured sub address areas and thus all Mem drivers will be synchronized and no parallel processing is done. Also note, that only one synchronization group can be configured for the multi binary jobs, that is parallel processing of multi binary jobs is not possible. For example use cases see also *Multi Binary* chapter.

5.2.3.1 Redirected requests



Caution

Please note that the redirected request use case can only be used if the Mem driver version is equal to AUTOSAR Version 22-11 or later. In versions prior to 22-11, the parameter Mem/MemGeneral/MemIndex did not exist, which is required for this feature.

If the redirected requests feature shall be used, that is if a satellite needs to forward memory access requests to the master, additional information must be configured both for master and satellite.

For the **master binary** a container `/MICROSAR/MemAcc/MemAccMultiBinaryConfiguration/MemAccMasterConfiguration/MemAccMultiBinarySynchronizationGroupConfiguration/MemAccRedirectedMemRequestConfiguration` must be configured for each satellite with redirect requests. Within this container the corresponding satellite's Id must be specified and the sector batches that shall be used must be referenced.

Note, that these sector batches for redirect requests can potentially be used both by the master and the satellite. However, in general the redirected Mem sector batches does not need to be referenced by a sub address area of the master. But it is important that the Mem instance that the redirected sector batches belong to is referenced in the relevant multi binary synchronization group, so that all multi binary accesses are synchronized. If this is not the case a dummy sub address area within a dummy address area, referencing the Mem with the redirected sector batches should be created and added to the relevant multi binary synchronization group. This is also verified by a validator.

The figure below shows the configuration of redirected request for the master binary. It focuses on the references between the multi binary configuration, the synchronization group configuration and the Mem sector batch configuration. For simplification the prefix MemAcc was omitted.

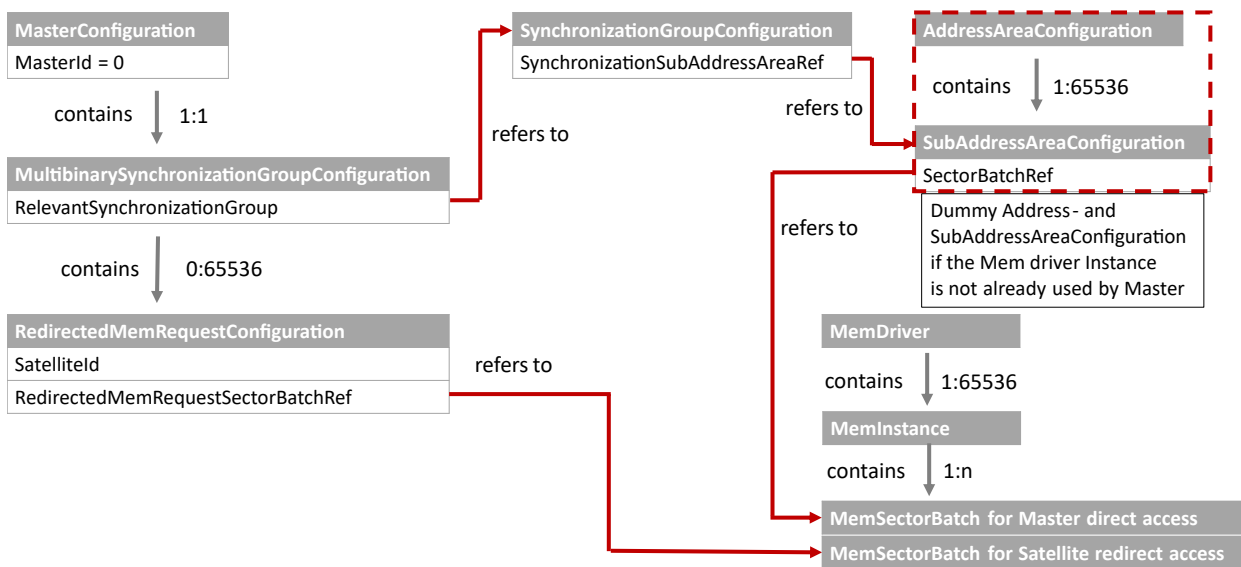


Figure 5.1 Configuration Overview: Redirect request in MemAcc Master

For the **satellite binary** that wants to redirect memory access requests to the master, the corresponding sub address areas must be referenced in `/MICROSAR/MemAcc/MemAccMultiBinaryConfiguration/MemAccSatelliteConfiguration/MemAccMultiBinarySynchronizationGroupConfiguration/MemAccRedirectedMemRequests`. These sub address areas must also be listed in the referenced relevant multi binary synchronization group which is checked by validation.

Note that these sub address areas must reference dummy Mem sector batches which should have the same AUTOSAR configuration as the real Mem sector batches referenced within the master (`/MICROSAR/MemAcc/MemAccMultiBinaryConfiguration/MemAccMasterConfiguration/MemAccMultiBinarySynchronizationGroupConfiguration/MemAccRedirectedMemRequestConfiguration`). The reason for this is, that the satellite is still responsible for splitting the memory access requests into Mem driver job steps and only forwards one job step to the master at a time. Therefore, the satellite must be aware of the Mem driver configuration.



Caution

The referenced dummy `MemSectorBatch` as well as the referenced real `MemSectorBatch` must have the same AUTOSAR parameters, e.g. value of `MemWritePageSize`, `MemEraseSectorSize` and so on must be the same. Also the `/Mem/MemGeneral/MemIndex` must be the same in both Mem configurations!

A dummy Mem for the satellite can be included in the DaVinci Configurator by Project Set-

tings -> AddModule -> Select from AUTOSAR Standard Definition.

The figure below shows a satellite configuration where requests targeting the SAA are redirected to the master.

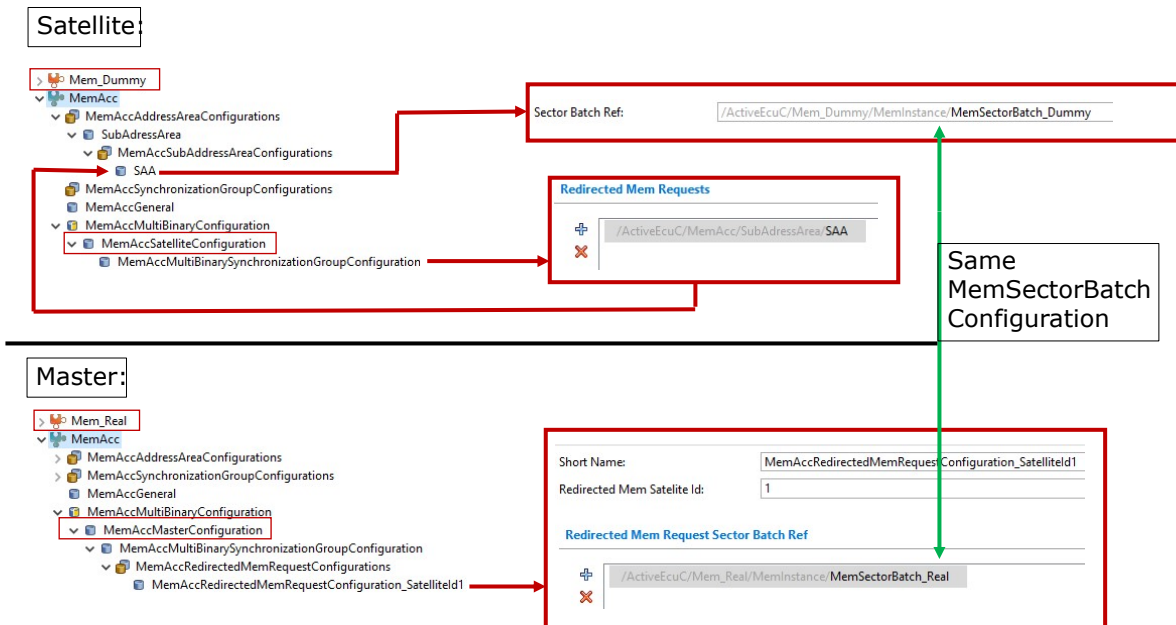


Figure 5.2 Configuration Overview: Redirect request in MemAcc Satellite

5.2.4 NAND-Flash Configuration

The configuration of NAND flash devices that should use bad block management is primarily done in the Mem configuration. In the MemInstance Configuration, set the `/Mem/MemInstance/MemBadBlockHandlingType` to `BBM_SKIP` and the `/Mem/MemInstance/MemNumberOfMaxBadBlocks` to the maximum amount of bad blocks that can occur on this MemInstance. If this value is 0, the bad block management is deactivated since no bad blocks can be processed.

All SubAddress areas, that target a MemInstance configured for bad block management via `/MemAcc/MemAccAddressAreaConfiguration/MemAccSubAddressAreaConfiguration/MemAccSectorBatchRef` will use the bad block management feature. Please note that one SubAddress area may not use the Multibinary feature and bad block management feature at the same time as mentioned in chapter NAND-Flash Memory Support.

In case at least one SubAddress area uses bad block management, the associated AddressArea has to configure an ErrorNotification in the parameter `/MemAcc/MemAccAddressAreaConfiguration/MemAccErrorNotificationName`.

5.2.5 Read only mode for specific memory areas

The configuration of the optional container `/MICROSAR/MemAcc/MemAccReadOnlyModeConfiguration` is described together with the background and usage within chapter *Read only mode for specific memory areas*.

5.2.6 Unaligned Read and Compare Support

By default, AUTOSAR requires the caller of `MemAcc_Read` and `MemAcc_Compare` to use source addresses and lengths that are aligned to the `MemMinReadSize` of the targeted memory device. The configuration parameter `/MICROSAR/MemAcc/MemAccGeneral/MemAccReadCompareAlignmentRequired` controls whether this requirement is enforced.

STD_ON (default)

The AUTOSAR alignment requirement is active. The caller of `MemAcc_Read` and `MemAcc_Compare` must ensure that the address and length are aligned to `MemMinReadSize`.

STD_OFF

The MemAcc accepts read and compare requests with arbitrary addresses and lengths and handles the alignment internally. The caller is not required to pre-align its read or compare requests.

Setting this parameter to `STD_OFF` has the following consequences that must be considered during integration:

- 1. Additional RAM required:**

A per-synchronization-group alignment buffer of size `MemMinReadSize` is generated.

- 2. Reduced read and compare throughput:**

Unaligned read and compare requests require additional `Mem_Read` operations to fulfil a single `MemAcc_Read` or `MemAcc_Compare` call, which increases the overall processing time.

It is therefore recommended to enable this option only when callers cannot guarantee alignment on their own.

6 Glossary and Abbreviations

Address Area

An address area is a logical area of memory. Upper layers only use logical addresses to access the address area. An address area contains at least one sub address area.

API

Application Programming Interface

AUTOSAR

Automotive Open System Architecture

BSW

Basis Software

DaVinci Configurator

Generation tool for MICROSAR Classic modules

DET

Development Error Tracer

DFLS

Data-Flash

EA

EEPROM Abstraction (Module of the AUTOSAR memory stack)

ECU

Electronic Control Unit

EcuM

ECU Manager

EEP

EEPROM

EXFLS

External Flash

FBL

Flash Bootloader

HSM

Hardware Security Module

ISR

Interrupt Service Routine

MICROSAR

Microcontroller Open System Architecture (the Vector AUTOSAR solution)

OTA

Over the air (Firmware update)

Page

Physical area of memory, smallest writable unit.

PFLS

Program-Flash

Physical Address

The physical addresses refer to the addresses which are provided by the memory driver.

RAM

Random Access Memory

RTE

Runtime Environment

Sector

Physical area of memory, smallest erasable unit.

Sub Address Area

Part of an address area, which corresponds to a physical continuous area of memory.

SWC

Software Component

SWS

Software Specification

7 Contact

Visit our website for more information on

- > News
- > Products
- > Demo software
- > Support
- > Training data
- > Addresses

www.vector.com